RAG Architecture — Summary

LegacyLens | Next.js, Supabase pgvector, Voyage Code 3, Claude | GnuCOBOL 3.2 Codebase Explorer

1. Vector DB Selection

Supabase (PostgreSQL + pgvector). Chosen for hybrid search in a single RPC call: pgvector cosine similarity + PostgreSQL full-text BM25, fused via Reciprocal Rank Fusion (RRF, k=50). Eliminates need for a separate keyword index (Elasticsearch). Unified data model — chunks, embeddings, query logs, and feedback in one database. Alternatives considered: Pinecone (no native BM25), Weaviate (overkill for ~5K chunks), ChromaDB (not production-grade), Qdrant (no full-text search).

2. Embedding Strategy

Voyage Code 3 (1024 dimensions). Code-specialized model trained on source code across languages. Supports asymmetric encoding (input_type="query" vs "document") which improves cross-modal retrieval (natural language → code). 78ms average query embedding latency. Chosen over OpenAI text-embedding-3 (general-purpose, no code specialization) and CodeBERT (requires self-hosting, smaller context window).

3. Chunking Approach

Language-specific chunkers that respect code boundaries. Target: 800 tokens, max: 1,500, min: 20, overlap: 3 lines.

- C files (.c): Function-level splitting. Regex detects definitions at column 0. Pre-function code becomes a "preamble" chunk. Over-size functions sub-split at blank lines.
- Headers (.h): Block-level splitting at #define, typedef, struct, enum boundaries (200+ token threshold).
- COBOL (.cob/.cbl): Division > Section > Paragraph hierarchy via column-aware regex (Area A detection).
- Bison/Yacc (.y/.l): Three-section split at %% delimiters, then per-grammar-rule within the rules section.
- Config (.conf/.def): Split at blank lines into macro_group or config_section chunks.

4. Retrieval Pipeline

Query → COBOL abbreviation expansion (26 terms) → TTL cache check (5min) → Voyage embedding (78ms) → Supabase RPC: BM25 + vector similarity → RRF fusion (k=50, 10 results) → Score normalization (0–100) → Top-N chunks to Claude (3 fast / 5 quality) → Streaming SSE with [Source N] citations.

BM25 handles exact identifiers (cob_move, PERFORM VARYING); vector search handles conceptual queries ("how does error handling work"). RRF gives fair weight to both without requiring cross-method score normalization. Context is formatted with absolute line numbers so the LLM can produce file:line citations.

5. Failure Modes

- Cross-file reasoning: Retrieves individual chunks, not call graphs. Multi-file tracing questions get incomplete answers.
- Macro-heavy code: Sees pre-expansion source. May retrieve the #define but not the code that invokes it, or vice versa.

- Very long functions (500+ lines): Sub-splitting can separate signature from core logic despite 3-line overlap.
- COBOL COPY statements: Copybook inclusions not resolved; queries about included data structures may miss field definitions.
- Ambiguous short queries: "MOVE" or "error" match too broadly. Abbreviation expansion helps but cannot fully disambiguate intent.
- Streaming timeout: Quality mode (Sonnet, 5 chunks) can approach the 60s Vercel limit on complex questions. A 55s safety cutoff returns a partial answer.

6. Performance Results

Search recall (20-query test set with known relevant files):

Match Count	Recall	Latency
3	45%	279ms
5	70%	159ms
10 (selected)	85%	158ms
15	90%	159ms
20	90%	167ms

LLM generation (Haiku, match_count=10, averaged over 20 queries):

Chunks	Latency	Input Tokens	Output Tokens	Answer Length
2	11.3s	2,793	639	5,653 chars
3 (fast)	13.4s	4,247	697	8,801 chars
5 (quality)	14.1s	6,537	775	13,893 chars
10	16.6s	13,174	853	29,372 chars

End-to-end: Fast mode 12–14s typical, Quality mode 15–22s. Cache hits return in <50ms. All API routes rate-limited (10–30 req/min per IP).